

Week 1 - Day 1b Supplementary: Strings and Booleans

Exercise 1: String Manipulation []

Write code that contains each letter of the alphabet **at most once** and prints the string in question. For example,

chechebsa -> "che" * 2 + "bsa"

1.1 banananight

```
# TODO: Your code here
```

1.2 abebe abebe abebe !

```
# TODO: Your code here
```

Exercise 2: Be the Computer []

2.1 Look at this code doing a computation with multiple steps,

```
x = 4
y = x * 2
z = y / (x - 2)
z
```

What is the value of `z`?

```
# TODO: Put your guess here before running the code cell
# TODO: Write code here to check your answer
# Run this cell after, to remove x, y, and z
del x, y, z
```

2.2 What happens if we swap the first two lines, like this?

```
y = x * 2
x = 4
z = y / (x - 2)
z
```

What is the value of `z`?

```
# TODO: Put your guess here before running the code cell
# TODO: Check your answer here
```

2.3 Given the code,

```
d = 1
c = d
b = c
a = b
a
```

What is the value of `a`?

```
# TODO: Put your guess here before running the code cell
# TODO: Check your answer here
```

2.4 Given the code,

```
x = 12
y = 5
z = y
y = 12
x = 1
z = x + y + z
z
```

What is the value of `z`?

```
# TODO: Put your guess here before running the code cell
# TODO: Check your answer here
```

2.5 Given the code,

```
exp = 10/2*4
exp = exp**2/4
exp
```

What is the value of `exp`?

```
# TODO: Put your guess here before running the code cell
# TODO: Check your answer here
```

Exercise 3: Computer Operations [] [] []

3.1 Consider a circle of diameter 5 and a rectangle with side lengths 2 and 4. Which one has the larger area? Use a comparison operator to find out.

Hint: The formula for the area of a circle is $\pi \left(\frac{\text{diameter}}{2} \right)^2$.

Note: Approximate π as 3.14

```
# TODO: Your code here
```

3.2 Now consider a circle of diameter `d` and a rectangle with side lengths `d - 1` and `d + 1`. Which one has the larger area? Does it depend on `d`?

```
# TODO: Your answer here
```

3.3 It is tedious to rewrite the comparison from the previous exercises repeatedly to replace the value of `d` everywhere. Instead, define an integer `d=5`, and write your comparison in terms of `d`. Now change the value of `d` and run your code again to explore the behavior of these areas.

```
# TODO: Your code here
```

What you have defined as `d` above is a *variable*. We did something similar when we used `exp` in Exercise 2.5. We will dive into variables more in the next exercise today!

Exercise 4: Physics Programs [] [] [] [] []

Let's say you roll a toy car off of a perfectly horizontal table at a speed of $v = 2 \text{ m/s}$. The table is $h = 1 \text{ m}$ tall measured from the ground. How far from the table would the car go before reaching the ground? Find this distance d ! You can assume gravity is 9.81 m/s^2

Hint: Gravity gives the *vertical* acceleration of the car as it is falling down. The car does not accelerate horizontally. Its initial *vertical* speed is 0 m/s , while its initial *horizontal* speed is 2 m/s .

4.1 Compute t , the time taken until the car hits the ground.

```
# TODO: Your code here
```

4.2 Using the value of t , now compute the distance d .

```
# TODO: Your code here
```

4.3 If you want to compute d for different values of h and v , you have to rewrite the computations from above. You can use variables to avoid that, as follows:

```
g = 9.81
h = 1
v = 2
```

Write the expression for t , the time taken until the car hits the ground, as an expression using g , h and v .

```
# TODO: Your answer here
```

4.4 We can save this computed value of t in a variable as well. Repeat the code above but assign your answer to a variable t .

```
# TODO: Your answer here
```

4.5 Add a line at the end with the computation for d (the distance the car travels before hitting the ground) in terms of t and the other variables.

```
# TODO: Your code here
```

4.6 Change the values of v and h and run your code from above again. Now you can solve this question for different values of v and h !

```
# TODO: Your code here
```

Exercise 5: String Formatting []

Abebe is a billionaire. As a billionaire he doesn't have much time to spend on reading things. He wants to look at his bank account and know how much money he has right away. Unfortunately, the bank's mobile app doesn't use commas to separate digits so we have to help him. Currently the bank's mobile application says he has $M=1123456789$ amount of money.

Can you write a few lines of code to make the amount of money to look like $1,123,456,789$? Make your code work for any billionaire who is not a trillionaire (in other words, assume that $1000000000 < M < 10000000000000$).

STUCK? FOLLOW THESE STEPS!

1. Convert the Number to a String

- You can't easily work with individual digits of a number.
- Convert the number into a string so you can slice it into parts.

Use `str(M)` to turn the number into a string.

2. Count the Number of Digits

- Once the number is a string, find out how many characters (digits) it has.

Use `length = len(s)` where s is the string version of the number.

≈ 3. Plan Where the Commas Go

Think about how you want to break up the digits based on the total length:

- If the string has **10 digits**, format it like: `1,234,567,890`
- If it has **11 digits**, format it like: `12,345,678,901`
- If it has **12 digits**, format it like: `123,456,789,012`

Use `if` statements: (You should not worry about `if` too much for now. We will visit it later again.) `if length == 10:`

□ 4. Use String Slicing to Extract Parts

- You can grab parts of a string using slice notation: `s[start:end]`
- Examples:
 - `s[0:2]` gives the first two digits
 - `s[4:7]` gives digits in the middle

Slice the string into 3-digit groups and connect them with commas.

□ 5. Use `+` to Combine the Parts

- Build the final result by combining your slices with commas.

Example: `python result = s[0:2] + "," + s[2:5] + "," + s[5:8] + "," + s[8:]`

```
# TODO: Your code here
```

Exercise 6: Extracting Digits [□□□□]

Recall the question from this morning's lab about extracting the lowest, second lowest, third lowest, etc... digit from the number 48756378291.

For each case, you had to write a new line of code, and if you wanted to do this for a different number, you would have to write new code again.

6.1 Define two variables, `n` and `k` as `n=48756378291` and `k=3`, and then write an expression using `n` and `k` that extracts the `k`-th lowest digit of `n`. When you change `n` and/or `k` and run your code again, do you always get the correct digit?

```
# TODO: Your code here
```

6.2 [□□□□] Again, define `n` and `k` as before, but this time write an expression that given `n` but with its `k`-th lowest digit set to 0.

Example, for `n=48756378291` and `k=3`, you should give `48756378091`.

Think of the number `n` as being made up of digits from right to left.

To remove the `k`-th lowest digit (i.e. the `k`-th digit from the right), follow these steps:

1. Use `10**(k-1)` to isolate the position of the digit you want to zero out.
2. Use `(n // 10**(k-1)) % 10` to find the actual digit in that position.

3. Multiply the digit back by $10^{(k-1)}$ to get its full value.
4. Subtract that value from n to zero out the digit.

```
# TODO: Your code here
```

6.3 In addition to n and k , this time also define a variable e , and write an expression that replaces the k -th lowest digit of n with e . as before, but this time write an expression that given n but with its k -th lowest digit set to 0.

Example: For $n=48756378291$ and $k=3$ and $e=7$, you should give 48756378791.

```
# TODO: Your code here
```

Exercise 7: Sneak peek into variables []

We briefly covered variables at the end of lecture. Don't worry if they were confusing: we will cover them in more detail tomorrow.

One very important detail: *programming variables* are NOT THE SAME as *mathematical variables*. Importantly, your last assignment to the variable until line 1 is the valid assignment for that variable at that line 1.

Guess what will happen if you run the following code blocks and check your answer in Jupyter.

7.1

```
x = 1
x

# TODO: Put your guess here before running the code cell

# TODO: Check your guess here
```

7.2

```
x = 1
x = 2
x

# TODO: Put your guess here before running the code cell

# TODO: Check your guess here
```

7.3

```
x = 1
y = x
y

# TODO: Put your guess here before running the code cell
```

```
# TODO: Check your guess here
```

7.4

```
x = 1  
y = x  
x = 2  
x
```

```
# TODO: Put your guess here before running the code cell
```

```
# TODO: Check your guess here
```

7.5:

```
x = 1  
y = x  
x = 2  
y
```

```
# TODO: Put your guess here before running the code cell
```

```
# TODO: Check your guess here
```

Congratulations 